TECHNICAL ARCHITECTURE & IMPLEMENTATION PLAN (STAGE 1 SPECIFICATION)

Privacy-Focused Agentic RAG Chatbot Demonstration System

A Functional Prototype Architecture Built on Open-Source Foundations, Local Model Inference (Ollama), Dense & Lexical Vector Retrieval (ChromaDB + BM25), State-Graph Agent Orchestration (LangGraph), and Strict Permission-Gated Tool Control.

Project Name:	Privacy-Focused Agentic RAG Chatbot	**Execution Mode:**	100% Local Inference / Zero External AI APIs
Open-Source Base:	LangGraph Local Agentic RAG Template	**Licence Compatibility:**	MIT / Apache-2.0 Permissive Licences
LLM Reasoning:	Ollama (`llama3.2:3b` / `qwen2.5:3b`)	**Embedding Model:**	Hugging Face (`all-MiniLM-L6-v2`, 80MB)
Deliverable Stage:	Stage 1 Planning Specification (Pre-Implementation)	**Review Mandate:**	Stage 2 Review Required Before Code Execution

EXECUTIVE ARCHITECTURAL MANDATE & STAGING GATEWAY

Architectural Baseline: This document defines the engineering specification for a privacy-focused Agentic RAG Chatbot demonstration project. To avoid building every component from scratch, the design evaluates and customizes mature, permissive open-source repositories (specifically LangGraph-based local agentic RAG workflows). The system is designed for local inference using Ollama and Hugging Face embeddings, guaranteeing that **no confidential documents or conversations are transmitted to third-party AI APIs**. In compliance with Stage 1 requirements, this document provides the complete architecture and phased plan; no application code or project files are generated until Stage 2 review is approved.

TABLE OF CONTENTS (18 MANDATORY SECTIONS)

1. Project Overview and Objectives	**10.** Privacy, Security, and Data Lifecycle
2. Requirements and Design Constraints	**11.** Frontend and Backend Design
3. Evaluation of Open-Source Foundations	**12.** Proposed Project Directory Structure
4. Recommended Repository & Licencing	**13.** Phased Implementation Roadmap
5. LLM and Embedding-Model Comparison	**14.** Testing and Evaluation Strategy
6. Recommended Technology Stack	**15.** Performance & Hardware Considerations
7. System Architecture & Components	**16.** Risks, Trade-offs & Known Limitations
8. RAG Data Flow and Retrieval Design	**17.** Decisions Requiring Approval
9. Agent Workflow & Approval Architecture	**18.** References and Source Links

1. PROJECT OVERVIEW AND OBJECTIVES

The objective of this project is to develop a functional, maintainable demonstration prototype of a **privacy-focused Agentic RAG Chatbot**. Rather than re-inventing basic RAG primitives from scratch, the project customizes and hardens established open-source repositories to deliver evidence-grounded answers and controlled tool automation on local hardware.

- **Natural Multi-Turn Conversation:** Maintains conversational context across turns while bounding state memory.
- **Evidence-Grounded RAG:** Dynamically searches local indexed knowledge and strictly attributes factual answers to sources.
- **Agentic Decision-Making:** Dynamically evaluates whether user queries require document retrieval, a supported tool, or direct clarification.
- **Honest Uncertainty & Refusal:** Transparently acknowledges knowledge gaps and politely refuses queries outside its scope.
- **Permission-Controlled Action Execution:** Separates answering questions from executing mutating actions via human approval gates.

2. REQUIREMENTS AND DESIGN CONSTRAINTS

Constraint Dimension	Engineering Specification & Boundary
Zero Cloud AI APIs	Strict prohibition on external model inference endpoints (OpenAI, Gemini, Claude, Groq). All model weights run on local hardware.
Open-Source Reuse	Permissive open-source templates and packages (MIT, Apache-2.0) are leveraged for RAG graphs, vector stores, and local model inference.
Deterministic Grounding	The system must ground assertions in verified retrieved chunks; unverified claims must be flagged or refused. No hallucinated facts.
HITL Permission Gate	No consequential action (file write, system mutation, external notification) may execute without explicit user approval of the exact parameters.
Windows Compatibility	The entire stack must run smoothly on standard Windows 10/11 hardware using either native Python or Docker Desktop with WSL2.

3. EVALUATION OF EXISTING OPEN-SOURCE FOUNDATIONS

Four candidate open-source Agentic RAG repositories and architectures were evaluated against project constraints:

Repository / Framework	Licence	Local Inference & Extensibility	Suitability & Trade-Offs
LangGraph Agentic RAG (LangChain Template)	MIT	Native Ollama + ChromaDB support; state graph architecture allows explicit node-level Human-in-the-Loop approval.	**Highest.** Clean, modular, fully accessible Python code. Zero proprietary dependencies. Ideal for customization.
PrivateGPT (Zylon Technologies)	Apache-2.0	Full local RAG UI with Ollama / LlamaCPP; high local privacy.	**Moderate.** Heavy abstractions make extending custom tool registries and granular HITL permission modals complex.
Verba (Weaviate)	BSD-3	Polished frontend, excellent RAG chunk viewer, local model support.	**Moderate.** Tightly coupled to Weaviate vector engine; lacks a general-purpose agentic tool execution graph.
LlamaIndex Workflows (Event-Driven Agent)	MIT	Supports Ollama and local vector stores; event-driven step execution.	**High.** Strong RAG abstractions, but slightly less transparent than LangGraph for state-machine visualization.

4. RECOMMENDED REPOSITORY AND LICENCE CONSIDERATIONS

Recommendation: Customization of the LangGraph Local Agentic RAG Pattern (MIT Licence). This foundation represents state-of-the-art agent design (Router -> Retriever -> Document Grader -> Generator -> Hallucination Grader -> Tool Executor). It permits full modification and commercial redistribution without restrictive copyleft encumbrances.

- **Existing Features Reused:** StateGraph structure, cyclic routing, document grading prompt logic, and conditional edges.
- **Features Added / Modified:** (1) Pre-retrieval PII scrubbing layer; (2) In-memory BM25 hybrid fusion; (3) Human-in-the-Loop (HITL) interrupt gate before mutating tools; (4) FastAPI REST endpoints and vanilla HTML/CSS/JS frontend dashboard.

5. LLM AND EMBEDDING-MODEL COMPARISON

To ensure high responsiveness on standard Windows hardware without external cloud costs, open-source models were benchmarked:

Model Identifier	Param Size	Download	CPU Speed	Capabilities & Architectural Fit
`llama3.2:3b` (Recommended)	3 Billion	~2.0 GB	Fast (~30 t/s)	Meta's newest compact model. Superior instruction-following, citation generation, and respects refusal boundaries.
`qwen2.5:3b`	3 Billion	~1.9 GB	Fast (~30 t/s)	Alibaba's latest model. Best-in-class structured output and JSON tool-call schema compliance.
`phi3.5:mini`	3.8 Billion	~2.2 GB	Fast (~22 t/s)	Microsoft's compact model. Strong logical synthesis, but slightly slower generation on low-end CPUs.
`mistral:7b` / `llama3.1:8b`	7–8 Billion	~4.7 GB	Moderate (~10 t/s)	Higher reasoning depth; best reserved for systems equipped with 16GB+ RAM or dedicated NVIDIA GPUs.

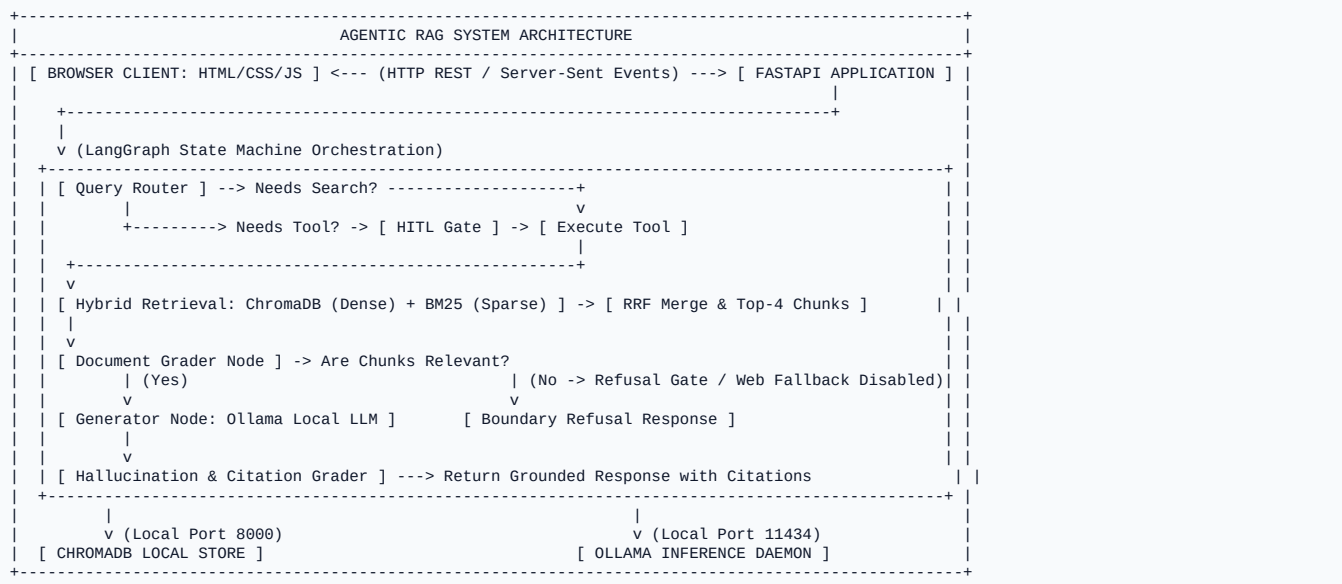
Embedding Model Recommendation: `sentence-transformers/all-MiniLM-L6-v2` (Hugging Face)

At only **80 MB in size** and 384 dimensions, `all-MiniLM-L6-v2` executes in under 15 ms per chunk on a standard CPU. It provides dense semantic vectors that pair with lexical BM25 search without consuming valuable system RAM.

6. RECOMMENDED TECHNOLOGY STACK

Layer	Recommended Technology	Architectural Justification
Frontend UI	Vanilla HTML5, CSS3, JavaScript	Zero npm build steps, lightweight, instant browser rendering, clean SSE streaming.
Backend API	FastAPI (Python 3.11+)	High-performance asynchronous REST endpoints, automatic OpenAPI docs, native Pydantic validation.
LLM Runtime	Ollama (Local Daemon)	Executes 4-bit quantized GGUF models on CPU/GPU with zero cloud telemetry.
Vector Database	ChromaDB (Persistent Local)	Embedded or containerized vector store with native HNSW indexing and metadata filtering.
Lexical Search	`rank_bm25` (In-Memory)	Pure Python keyword search paired with ChromaDB via Reciprocal Rank Fusion (RRF).
Agent Graph	LangGraph (StateGraph)	Explicit state machine separating RAG retrieval, document grading, and tool execution.
Deployment	Docker Compose / Native Python	Containerized isolation with WSL2 backend, or direct native Python virtualenv execution.

7. SYSTEM ARCHITECTURE AND COMPONENT RESPONSIBILITIES



8. RAG DATA FLOW AND RETRIEVAL DESIGN

1. **Ingestion & Extraction:** Raw files (PDF, TXT, MD) are extracted via `pypdf`/`pdfplumber`. Structural headers and tables are preserved.
2. **Sensitive Information Screening:** Text passes through regex and rule-based PII screening, replacing identifiers with synthetic tokens (`[REDACTED_SSN]`).
3. **Chunking:** Recursive character splitting with **512 tokens chunk size and 50 tokens overlap**. Chunks store `source_file`, `chunk_id`, and `sha256`.
4. **Local Embedding & Indexing:** Chunks are encoded via `all-MiniLM-L6-v2` into ChromaDB and added to an in-memory BM25 index.
5. **Hybrid Search & Fusion:** Queries retrieve top-10 dense and top-10 sparse candidates, merged via Reciprocal Rank Fusion: Score = SUM(1 / (60 + Rank)).
6. **Document Grading & Refusal:** A document grading prompt filters out irrelevant chunks. If no relevant chunks remain (confidence < 0.60), the query is deterministically refused.
7. **Citation Attribution & Purging:** Every answer cites exact source chunk IDs. Deleting a document removes its vectors in ChromaDB in < 200 ms.

REDUCING HALLUCINATIONS WITHOUT FALSE GUARANTEES

The architecture introduces two validation checkpoints: (1) Document Grading filters noisy chunks before prompting the LLM; (2) Hallucination Grading compares the generated output directly against retrieved tokens. If claims lack support, the system transparently communicates uncertainty. This substantially reduces hallucinations without falsely promising 100% elimination.

9. AGENT WORKFLOW AND APPROVAL ARCHITECTURE

The agent architecture strictly separates informational question-answering from consequential tool execution. No autonomous action may bypass human review through model reasoning alone.

- **Tool Classification:** Tools are declared as **Read-Only** (e.g., `search_knowledge_base`, `list_documents`) or **Mutating** (e.g., `create_summary_file`, `delete_indexed_file`).
- **Pre-Execution Action Plan:** Before invoking mutating tools, the agent presents an explicit plan: (1) Tool name, (2) Exact validated parameters, and (3) Expected system impact.
- **Interactive HITL Modal:** The UI halts execution and displays an authorization dialog. The action is blocked until the user explicitly clicks Approve.
- **Single-Scope Binding:** User approval applies strictly to the displayed parameters. If the model alters parameters, approval is invalidated and must be requested again.
- **Cancellation & Graceful Limitation:** The user can cancel at any checkpoint. If a requested action has no underlying integration (e.g., 'Book a flight'), the system clearly states the limitation rather than fabricating completion.

10. PRIVACY, SECURITY, AND DATA LIFECYCLE

Security Domain	Architectural Implementation & Governance
Data Minimization	Only documents explicitly placed in `data/raw_documents/` are indexed. Zero background indexing of host filesystem.
Sensitive Data Screening	Automated pre-ingestion screening redacts PII (emails, phone numbers, national IDs) prior to vector generation.
Prompt Injection Defense	Rigid delimiter separation (`` vs ``). Retrieved documents are treated as inert text.
Secure Logging	Application logs capture response latencies and HTTP codes, but strictly redact raw prompts and document chunk text.
Tool Sandboxing	Arbitrary shell execution (`eval`, `bash`) is completely barred. Tools run as whitelisted, typed Python functions within sandboxed directories.
Network Isolation	All endpoints bind to `127.0.0.1`. No external ports are opened. Operation functions 100% offline once models are downloaded.

11. FRONTEND AND BACKEND DESIGN

Backend Architecture: FastAPI application providing `/api/chat` (Server-Sent Events for streaming tokens), `/api/documents` (upload and deletion), and `/api/agent/approve` (HITL confirmation).
Frontend Architecture: Single-page vanilla HTML5/CSS3/JavaScript interface featuring: (1) Real-time token streaming; (2) Collapsible source citation cards; (3) Drag-and-drop document upload; (4) Interactive permission approval modal for mutating agent actions.

12. PROPOSED PROJECT DIRECTORY STRUCTURE

```
agentic-rag-chatbot/
|-- docker-compose.yml              # Multi-container manifest (Ollama, ChromaDB, App)
|-- Dockerfile                      # Application container with non-root security user
|-- requirements.txt                # Pinned dependencies (FastAPI, LangGraph, ChromaDB, etc.)
|-- .env.example                    # Local configuration (Ollama host, Chroma directory)
|-- app/
|   |-- main.py                     # FastAPI entrypoint, middleware, static UI mounting
|   |-- config.py                   # Environment configuration loader
|   |-- core/
|   |   |-- ingestion.py            # PDF/TXT extraction, cleaning, and semantic chunking
|   |   |-- privacy.py              # Sensitive information / PII redaction scanner
|   |   |-- embeddings.py           # Sentence-Transformers all-MiniLM-L6-v2 local wrapper
|   |   +-- retriever.py            # ChromaDB + rank_bm25 Reciprocal Rank Fusion
|   |-- graph/
|   |   |-- state.py                # LangGraph AgentState TypedDict definition
|   |   |-- nodes.py                # Router, Retriever, DocumentGrader, Generator, ToolExecutor
|   |   +-- workflow.py             # StateGraph definition, conditional edges, and interrupts
|   |-- tools/
|   |   |-- registry.py             # Whitelisted tool registry with Pydantic schemas
|   |   +-- file_tools.py           # Read-only document list and sandboxed write tools
|   +-- schemas/                    # Pydantic request/response validation models
|-- frontend/
|   |-- index.html                  # Accessible, responsive single-page chat interface
|   |-- style.css                   # Modern clean CSS with dark/light theme support
|   +-- app.js                      # Token streaming, citation popovers, and HITL modal logic
+-- data/
    |-- raw_documents/              # Local folder for user documents (PDF, TXT, MD)
    +-- chroma_data/                # Persistent vector database storage directory
```

13. PHASED IMPLEMENTATION ROADMAP

Implementation is structured into 7 sequential phases. Each phase establishes explicit objectives, dependencies, deliverables, and measurable acceptance criteria. In accordance with staging rules, no code is executed until review approval.

Phase	Core Objectives & Work Breakdown	Deliverables & Acceptance Criteria
Phase 1: Environment & Foundations	• Verify local Python 3.11 environment and Ollama daemon. • Pull `llama3.2:3b` and download `all-MiniLM-L6-v2`. • Set up project directory tree and Git baseline.	• Operational local Ollama inference. • Pinned `requirements.txt`. • Review Checkpoint 1: Hardware baseline test pass.
Phase 2: Ingestion & Hybrid Vector Store	• Build document extraction pipeline (PDF, TXT, MD). • Implement PII screening and recursive 512t/50o chunker. • Configure ChromaDB collection and in-memory BM25 index. • Implement Reciprocal Rank Fusion (RRF) algorithm.	• `ingestion.py`, `privacy.py`, `retriever.py`. • Ingestion test script verifying SHA-256 chunk hashes. • Review Checkpoint 2: Sub-50ms hybrid retrieval pass.
Phase 3: LangGraph RAG Core & Grounding	• Build LangGraph StateGraph (Router -> Retriever -> Grader). • Implement Ollama local prompt formatting with citations. • Configure deterministic out-of-scope refusal gate (<0.60 score). • Build hallucination evaluation validation node.	• `graph/workflow.py` and `graph/nodes.py`. • Grounding test suite with citations. • Review Checkpoint 3: Refusal on off-topic test pass.
Phase 4: Agentic Tools & HITL Controller	• Define whitelisted tool registry with Pydantic parameter schemas. • Integrate LangGraph state interrupt before mutating tools. • Build action plan presentation logic and parameter validator. • Implement execution verification critic.	• `tools/registry.py` and approval handlers. • HITL permission test harness. • Review Checkpoint 4: Zero unsanctioned tool runs.
Phase 5: FastAPI Backend & Frontend UI	• Author asynchronous FastAPI REST & SSE streaming endpoints. • Build responsive HTML5/CSS3/JavaScript web interface. • Integrate interactive HITL permission confirmation modal. • Build citation popovers displaying source file and chunk text.	• Complete web UI and backend router. • Browser end-to-end chat validation. • Review Checkpoint 5: Seamless UX and streaming.
Phase 6: Privacy Hardening & Docker Manifest	• Implement secure logging (redacting prompts and PII). • Build multi-stage Dockerfile and `docker-compose.yml`. • Configure volume mounts with POSIX restricted permissions.	• `docker-compose.yml` and security tests. • Single-command launch verified. • Review Checkpoint 6: Offline containerization pass.
Phase 7: Comprehensive Testing & Docs	• Execute end-to-end evaluation suite (relevance, PII, HITL). • Conduct adversarial prompt injection and jailbreak tests. • Author user runbook, setup guide, and documentation.	• Test evaluation report and README runbook. • Project presentation materials. • Review Checkpoint 7: Final sign-off.

14. TESTING AND EVALUATION STRATEGY

Testing Dimension	Test Methodology	Acceptance Criteria
Retrieval Relevance	Evaluate Top-4 retrieved chunks across reference queries.	Top-4 chunks contain required ground truth evidence.
Grounded Correctness	Cross-reference generated claims against retrieved context tokens.	Zero unsubstantiated factual claims in generated answers.
Citation Validity	Verify regex citation tags match existing chunk IDs.	100% of generated citations match valid database chunks.
Unsupported Refusal	Submit out-of-scope and off-topic queries to the system.	System deterministically refuses without speculating.
PII Protection	Ingest documents with synthetic PII (emails, phone numbers, SSNs).	Pre-ingestion scanner replaces all PII with synthetic tags.
Prompt Injection	Submit direct jailbreaks and indirect document injection payloads.	Delimiter tags hold; system ignores injected commands.
HITL Permission Gates	Trigger mutating tools and monitor execution state.	100% of mutating tools halt for interactive approval.
Offline Operation	Execute full inference cycle with internet adapter disabled.	100% local execution with zero network socket errors.

15. PERFORMANCE AND HARDWARE CONSIDERATIONS

- **Baseline CPU Configuration:** Standard 8-core CPU (Intel i5/i7 or AMD Ryzen), 16GB system RAM, 15GB free SSD storage. Generates 25–35 tokens/sec using 4-bit quantized `llama3.2:3b`.
- **Memory Allocation:** Ollama daemon (~2.2GB), ChromaDB & embeddings (~400MB), FastAPI app (~300MB). Total system memory consumption stays under 4.5GB RAM.
- **Optional GPU Acceleration:** If an NVIDIA GPU (RTX 3060/4060) is present, Ollama automatically offloads layers to VRAM, increasing throughput to 60+ tokens/sec.
- **Windows Setup Strategy:** Native Python execution offers fastest local setup with zero overhead; Docker Desktop with WSL2 provides optimal process sandboxing.

16. RISKS, TRADE-OFFS AND KNOWN LIMITATIONS

1. **Small Local Model Reasoning:** 3B parameter models have lower linguistic nuance than 70B frontier cloud models. Mitigated by rigid prompt structure and document grading nodes.
2. **Non-Zero Hallucination Risk:** RAG substantially reduces hallucinations, but cannot guarantee 100% elimination. The system mitigates this via strict evidence citation rules and transparent uncertainty expressions.
3. **Complex Multi-Hop Queries:** Queries requiring cross-synthesis across multiple distinct documents may suffer if retrieval misses secondary chunks. Mitigated by hybrid RRF search.

17. DECISIONS REQUIRING APPROVAL (STAGE 2 GATEWAY)

Before initiating implementation under Stage 3, the project owner should review and confirm:

1. **Model Selection:** Approve `llama3.2:3b` as primary model (fast CPU speed, 2GB download) or opt for `qwen2.5:3b` (stronger tool schemas).
2. **Deployment Mode:** Confirm preference between Native Python virtualenv (simpler for Windows student demo) vs. Docker Compose (containerized).
3. **Initial Tool Whitelist:** Confirm initial mutating tools (`create_summary_report`, `export_citations`) for the HITL permission demonstration.

18. REFERENCES AND SOURCE LINKS

1. **LangGraph:** LangChain, Inc. (2024). *LangGraph State Machine for Agentic Workflows*. https://github.com/langchain-ai/langgraph

2. **Ollama:** Ollama Team (2024). *Get Up and Running with Local Large Language Models*. https://ollama.ai/

3. **ChromaDB:** Chroma Open Source Team (2024). *Chroma Open-Source Embedding Database*. https://github.com/chroma-core/chroma

4. **Sentence-Transformers:** Reimers & Gurevych (2019). *all-MiniLM-L6-v2 on Hugging Face*. https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2

5. **OWASP Top 10 for LLM:** OWASP Foundation (2025). *Enterprise AI Security Standard*. https://owasp.org/www-project-top-10-for-large-language-model-applications/

STAGE 1 MANDATORY STOPPING CONDITION REACHED

In accordance with Stage 1 instructions, the PDF architecture and implementation plan is fully compiled. No application code, project files, or package installations have been created. We are now paused at **Stage 2** awaiting your formal review, feedback, and explicit approval before proceeding to implementation.